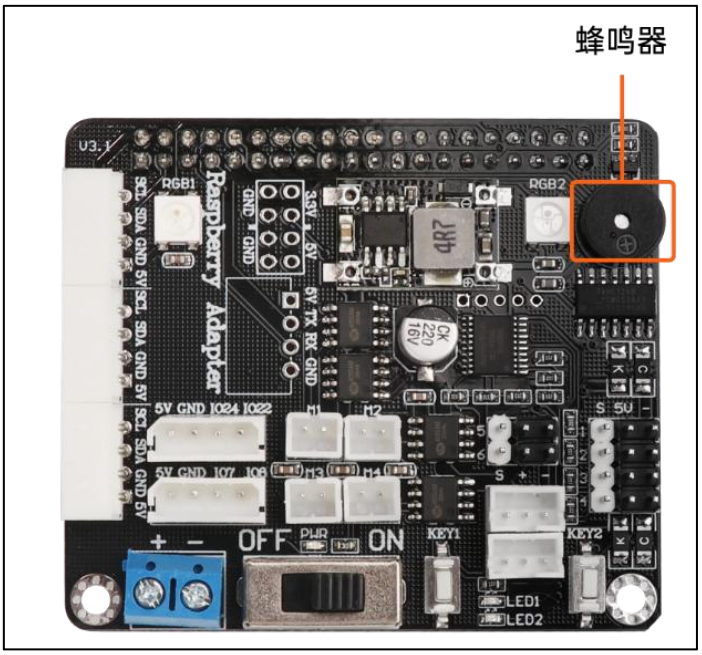
第13课 控制蜂鸣器

1.准备工作

树莓派扩展板上板载了一个蜂鸣器，如下图所示位置：



2.实现原理

本节课的整体实现思路如下所示：

ArmPi FPV扩展板采用的是有源蜂鸣器，只需通过设置电平高低，即可实现对蜂鸣器的发声控制。

该程序的源代码位于：**/home/ubuntu/armpi_fpv/src/sensor/scripts/sensor_control.py**

```
# 蜂鸣器驱动
def setBuzzer(msg):
    GPIO.setup(6, GPIO.OUT)
    GPIO.output(6, 1)
    rospy.sleep(msg.data)
    GPIO.setup(6, GPIO.OUT)
    GPIO.output(6, 0)

if __name__ == '__main__':
    rospy.init_node('sensor')

    rgb_led = rospy.Subscriber("/sensor/rgb_led", Led, setRGB)
    buzzer_sub = rospy.Subscriber("/sensor/buzzer", Float32, setBuzzer)
    motor_sub = rospy.Subscriber("/sensor/motor", Motor, setMotor)
    servo_sub = rospy.Subscriber("/sensor/servo", Servo, setPWMServoPulse)
    battery_pub = rospy.Publisher('/sensor/battery', Int32, queue_size=1)
    rospy.sleep(0.2)
```

蜂鸣器的控制主要用到GPIO模块中的三个函数：setmode()、setup()、output()。其中：

setmode()函数用于设置GPIO口的编码方式，程序中采用是BCM编码方式；

setup()函数用于设置GPIO口的工作模式，由于程序需要控制蜂鸣器的电平，故将模式设置为输出。在扩展板上，蜂鸣器的控制端口对应的BCM编码为6，故该函数的参数设置为(6,GPIO.OUT)。

output()函数用于设置GPIO口的输出电平，第1个参数“6”为蜂鸣器的控制端口号，第2个参数为电平状态，其中“1”表示高电平，“0”表示低电平。

3.实验步骤

1) 打开 LX 终端。点击桌面左下角的“**Applications**”图标，然后选择“**Terminal Emulator**”打开 Ubuntu 的命令行终端。

2) 在打开的界面中，输入指令“**rostopic pub /sensor/buzzer**”，再通过 TAB 键补齐指令。

3) 接着，修改指令中的“**data:0.0**”（比如，我们将 0.0 改为 0.5），按下回车运行程序。

4) 如需关闭此程序，可按下“**Ctrl+C**”。

4.功能实现

程序启动后，扩展板上的蜂鸣器将会发声0.5秒，然后自动关闭。